

Examen réparti 2

11 janvier 2022

Antoine Miné

Déroulement de l'examen

L'examen dure 2h.

Tous les documents papier sont autorisés (impressions des transparents, notes personnelles, livres, etc.). L'utilisation de moyens de communication (téléphone, tablette, etc.) est interdite. L'utilisation d'une clé USB est autorisée, en lecture seule. Seul le format FAT est supporté (pas NTFS).

Vous trouverez sur votre compte dans le répertoire de travail `eclipse-workspace` un projet Eclipse `ILP4` contenant les sources d'ILP4. Vous complèterez ce projet en répondant aux questions de l'énoncé. En cas de problème, une archive ZIP des sources d'ILP4 est également disponible sur le site de l'UE : <http://www-master.ufr-info-p6.jussieu.fr/site-annuel-courant/DLP>.

Tous vos fichiers doivent être rédigés dans le package `com.paracamplus.ilp4.exam2021` et ses sous-packages du projet Eclipse `ILP4`. Ils se trouveront donc dans un sous-répertoire de `eclipse-workspace/ILP4/Java/src/com/paracamplus/ilp4/exam2021` de votre compte. Placez bien les fichiers dans le package (et donc dans le répertoire) demandé pour chaque question.

En fin d'examen, sauvegardez tous vos fichiers, fermez Eclipse et déloguez-vous. Les fichiers de votre compte seront automatiquement ramassés. L'examen sera corrigé à la main. Quand du code est demandé, les erreurs de syntaxe ne seront pas pénalisées. Il est souhaitable de n'expliciter que les parties du code qui vous semblent importantes, et de remplacer les parties peu informatives par un commentaire indiquant succinctement ce qui a été omis (inutile, par exemple, le détailler du code répétitif dans son intégralité).

Mise en route

- Ouvrez un navigateur et allez à l'URL : <http://www-master.ufr-info-p6.jussieu.fr/site-annuel-courant/DLP>. Vous y trouverez notamment les transparents du cours. Une copie locale de la documentation Java est disponible sur <https://www-ppti.ufr-info-p6.jussieu.fr>.
- Lancez Eclipse en tapant dans un terminal : `eclipse-2020 &`
- Ouvrez le projet `ILP4` contenant les sources d'ILP4 et vérifiez qu'elles compilent bien. Si le projet `ILP4` n'apparaît pas dans le « Package Explorer », ouvrez le menu « File / Open Projects From Filesystem... », cliquez sur « Directory... », choisissez le répertoire `eclipse-workspace/ILP4`, puis « Finish ».
- Si le plug-in ANTLR ne compile pas automatiquement les fichiers de grammaires `.g4` en sources `.java`, vous pouvez taper dans un terminal les commandes suivantes :

```
cd ~/eclipse-workspace/ILP4
./compile_ANTLR.sh
```

Dans Eclipse, cliquez sur le projet dans le « Package explorer » et tapez F5 (ou cliquez sur « Refresh »). Les sources générées par ANTLR 4 devraient apparaître dans `targets/generated-sources/antlr4/antlr4`. Il faut attendre quelques secondes avant qu'elles soient compilées par Eclipse.

Après chaque modification d'un fichier `.g4`, vous devrez à nouveau lancer la commande `./compileANTLR.sh` depuis le répertoire `eclipse-workspace/ILP4`.

- Une cause possible d'erreur de compilation est un environnement Java mal configuré. Faites un clic droit sur le nom du projet dans le « Package Explorer », cliquez sur « Properties », puis vérifiez les éléments suivants :
 - La version de la bibliothèque système (onglet « Java build path / Libraries » puis « JRE System Library ») doit être 8 ou supérieure. Vous pouvez la changer avec le bouton « Edit... / Alternate JRE ». Les JDK disponibles localement sont dans le répertoire `/usr/lib/jvm`.
 - JUnit 4 doit être présent. Sinon, ajoutez-le avec le bouton « Add Library ».
 - Le niveau de compatibilité Java (« Java compliance level ») dans l'onglet « Java Compiler » des propriétés doit correspondre à la version du JDK utilisée.
 - La liste des répertoires source doit contenir le répertoire `target/generated-sources/antlr4` (dans « Java build path », onglet « Source »).

1 Définition de listes par compréhension

Dans cet examen, nous ajoutons à ILP4 une construction pour *définir des listes par compréhension*. Nous nous inspirons d'une construction similaire existant en Python. Voici un exemple en ILP :

```
let l = [ 2 * x for x in range(5) ] in
print(l)
```

Ce programme a pour effet de construire la liste de 5 éléments obtenue en évaluant l'expression $2 * x$ pour les valeurs successives de x de 0 (inclus) à 5 (exclu). La liste est affectée à la variable locale l , puis affichée. Le résultat attendu à l'écran est donc la chaîne suivante :

```
[0, 2, 4, 6, 8]
```

Comme en Python, la construction a une clause optionnelle *if expr*. Si elle est présente, elle agit comme un filtre : l'expression *expr* est testée pour chaque valeur de x et un élément est ajouté à la liste pour cette valeur de x uniquement quand *expr* est vraie. Ainsi :

```
let y = 3 in
let l = [ 2 * x for x in range(5) if x >= y ] in
print(l)
```

ne gardera que les valeurs de $2 * x$ pour les valeurs de x entre 0 (inclus) et 5 (exclu) quand $x \geq y$ est vrai, donc pour x valant 3 et 4. Le résultat attendu est donc : « [6, 8] ». Dans ce cas, la liste résultante a une taille de 2, inférieure à la valeur 5 précisée dans la construction par `range(5)`, puisque certaines valeurs de x sont ignorées par la clause *if*.

La forme générale de l'expression qui retourne une liste définie par compréhension est :

```
[ body for var in range(max) if cond ]
```

où *body*, *max* et *cond* sont des expressions, *var* est un identifiant de variable, tandis que *for*, *in*, *range* et *if* sont des mots-clés. La clause *if cond* est optionnelle et peut être omise. Cette expression est évaluée de la manière suivante :

1. l'expression *max* est évaluée ; si sa valeur n'est pas un entier, une exception est signalée ;
2. une liste l initialement vide est créée ;
3. pour chaque valeur entière i de 0 (inclus) à la valeur de *max* (exclue) :
 - (a) si la clause *if cond* est présente, alors l'expression *cond* est évaluée dans un environnement où la variable *var* vaut i ;
 - (b) si la valeur de *cond* n'est pas le booléen faux, ou si la clause *if cond* est absente, alors l'expression *body* est évaluée dans un environnement où la variable *var* vaut i et la valeur trouvée est ajoutée à la fin de la liste l ;
sinon (i.e., *cond* est présente et s'évalue en faux), *body* n'est pas évaluée et la liste l n'est pas modifiée pour cette valeur de i ;
4. l'expression retourne la liste l .

L'ajout de cette construction implique d'ajouter une nouvelle structure de données, les *listes*, qui n'existent pas nativement dans ILP. Nous demandons également d'ajouter une primitive `get(list, i)` pour retourner l'élément à la position i de la liste *list*, où i et *list* sont des expressions. Elle signale une exception si l'évaluation de *list* ne donne pas une liste, ou si celle de i ne donne pas un entier positif et strictement inférieur à la taille de la liste. Enfin, l'affichage d'une liste avec la primitive `print` existante en ILP sera de la forme « [elem1, elem2, ..., elemN] » (les éléments de la liste, séparés par des virgules « , » et encadrés de crochets « [», «] »). Les listes sont immuables : une fois créées, il est impossible d'ajouter des éléments ni de les modifier. Nous ne demandons pas d'implanter de primitive pour retourner la taille de la liste.

2 Grammaire et analyse syntaxique

Grammaire (2 pts). Écrivez une grammaire qui étend ILP4 avec la construction suivante définissant les listes par compréhension :

```
[ body for var in range(max) if cond ]
```

où *body*, *max* et *cond* sont des expressions, *var* est un identifiant de variable et la partie *if cond* est optionnelle. Cette grammaire se trouvera dans le fichier `eclipse-workspace/ILP4/Java/src/com/paracamplus/ilp4/exam2021/grammar/ILPgrammarExam2021.g4`. N'oubliez pas de lancer `./compileANTLR.sh` pour compiler la grammaire en Java.

Arbre syntaxique (2 pts). Proposez une extension de l'arbre syntaxique pour gérer cette nouvelle construction. Vous fournirez toutes les interfaces correspondantes dans le package `com.paracamplus.ilp4.exam2021.interfaces` et toutes les classes dans le package `com.paracamplus.ilp4.exam2021.ast` sans oublier celles liées aux motifs fabrique et visiteur.

Analyse syntaxique (2 pts). Implantez dans le package `com.paracamplus.ilp4.exam2021.parser` les classes `ILPMLparser` et `ILPMLlistener` implantant l'analyseur syntaxique pour le langage étendu avec cette construction.

3 Interprétation (4 pts)

Implantez dans `com.paracamplus.ilp4.exam2021.interpreter` un interprète qui enrichit ILP4 avec :

1. la construction `[ body for var in range(max) if cond ]` pour définir des listes par compréhension,
2. la primitive `get(list, i)` pour retourner l'élément à l'indice *i* de la liste *list*.

Pour représenter les listes ILP dans l'interprète, vous pourrez simplement utiliser le type Java `List<Object>`. Ce type définit une méthode `toString` qui assure que la primitive `print` existante en ILP affichera la liste sous la forme demandée.

4 Tests (2 pts)

Proposez une base de tests dans le répertoire `eclipse-workspace/ILP4/Java/src/com/paracamplus/ilp4/exam2021/samples/`. Vous fournirez au moins 4 tests *qui ne reprennent pas les exemples de l'énoncé* et qui testent les cas suivants :

1. la construction d'une liste sans clause `if cond`;
2. la construction d'une liste avec clause `if cond`;
3. la primitive `get` ;
4. un cas d'erreur (lors de la construction de la liste ou lors d'un appel à `get`).

Pour chaque test, vous fournirez le source ILPML ainsi que les fichiers permettant de vérifier les résultats attendus.

Vous fournirez également un fichier texte `eclipse-workspace/ILP4/Java/src/com/paracamplus/ilp4/exam2021/samples/tests.txt` précisant, pour chaque test, le comportement ou l'erreur testé. La classe `InterpreterTest` n'est pas demandée.

5 Compilation

Arbre syntaxique (1 pt). Donnez la version ASTC pour la construction d'une liste par compréhension. Vous fournirez le code Java de toutes les interfaces correspondantes dans le package `com.paracamplus.ilp4.exam2021.compiler.interfaces` et de toutes les classes dans le package `com.paracamplus.ilp4.exam2021.compiler.ast`. L'interface de visiteur et la fabrique des ASTC ne sont pas demandées.

Analyse des variables libres (2 pts). Donnez dans le package `com.paracamplus.ilp4.exam2021.compiler` le code de la classe qui effectue l'analyse des variables libres. L'analyse des variables globales n'est pas demandée.

Bibliothèque d'exécution (3 pts). Dans le répertoire `eclipse-workspace/ILP4/Java/src/com/paracamplus/ilp4/exam2021/C`, donnez la bibliothèque d'exécution C enrichie pour supporter dans le compilateur :

1. la structure de données des listes ;
2. la primitive `get`.

Schéma de compilation (3 pts). Donnez un schéma de compilation détaillé pour la construction des listes par compréhension. Vous fournirez votre réponse dans un fichier texte : `eclipse-workspace/ILP4/Java/src/com/paracamplus/ilp4/exam2021/compiler/schema.txt`. Le code de la classe `Compiler` n'est pas demandé.